GMI German-Malaysian Institute

DIPLOMA PROGRAMME

MINI PROJECT

Academic Period : Jan 2026

Program	:	**DIPLOMA IN SOFTWARE ENGINEERING**
Course	:	**TEST AUTOMATION AND CONFIGURATION**
Code	:	**STT 2273**
Duration	:	**4 WEEKS**
Total Marks	:	**60 MARKS**

Title : **PUPPETEER**

Student's Name : 2. NUR FATIN FASISHAH BINTI M.RUSDEI@RUSDI
(Individual / Group)

TTO's Name :
Semester/Trade/Group :
Date :

STT2273 TEST CONFIGURATION & MEASUREMENT

STT 2273 Test Automation & Configuration Revision Number: Rev 00/2024 Semester: JAN 2026

Assessment Specification Table (SWE)

Mini project																				
Raw Marks:			60																	
Percentage (in Doc 01):			20																	

No	Assessment Criteria	Full Marks	Score (if applicable)	Student's Marks	CLO	Level of Difficulty	DK, DP, NA (ETAC Programme Only)	Comments	PLO SWE 9 PLO - Start Intake July 2025	PLO SWE 11 PLO - Intake July 2023 - Jan 2025
1	Tool Introduction	5			3	C2			3	2
2	Tool Requirement	10			3	C2			3	2
3	Tools Advantage/Disadvantage	10			3	C2			3	2
	Presentation Criteria									
4	Tools Installation and Setup	10			3	C3			3	2
5	Tools Setup	10			3	C3			3	2
6	Usage Examples	15			3	C3			3	2
	Total Full Marks	60	Total Student's Marks							

SUMMARY

Note:
FM - Full Marks,
StM - Student's Mark

Total

	Raw Marks		Percentage			Raw Marks		Percentage			Raw Marks		Percentage	
CLO	FM	StM	FM	StM	PLO	FM	StM	FM	StM	PLO	FM	StM	FM	StM
CLO 1	0		0.0		PLO 2	0		0.0		PLO 3	0		0.0	
CLO 2	0		0.0		PLO 3	60		20.0		PLO 2	60		20.0	
CLO 3	60		20.0		PLO 3	60		20.0		PLO 2	60		20.0	
CLO 4	0		0.0		PLO 5	0		0.0		PLO 4	0		0.0	
CLO 5	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
CLO 6	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
Total Marks	60		20.0											

Signature

( )

STT2273 TEST CONFIGURATION & MEASUREMENT

MINI PROJECT
60 MARKS
GROUP OF 3/4

INSTRUCTIONS

1. Find 1 tools/software/system/any new technology related with Automation Testing / Version Control (may run on any platform such as Windows/Linux/Desktop/Server) with free/open source license. The chosen tools must not our lab practical tools which are Selenium & GitHub. In the same class, no one can choose the same tools. Content of the report and slides are listed below:

2. **Introductions**.
3. **Requirements** to use the tools.
4. **Advantage/disadvantage** of the tools. (May compare with other tools)
5. Procedure of tools **Installation and Setup**.
6. Tools **Features**.
7. **Screenshots** usage example with caption (In Report only).
8. **Demonstration** (During presentation).
9. **Conclusion**.

* Presentation date is **2 May 2026**, please submit your report before the presentation day.

Example of Automation Testing tools:

1. iMacros	2. QA Wolf	3. Testia Tarantula	4. Appium
5. The Grinder	6. Tsung	7. Testerum	8. Puppeteer

Example of Version Control tools:

1. Concurrent Versions System (CVS)	2. Apache Subversion (SVN)	3. Mercurial	4. Monotone
5. Revision Control System (RCS)	6. GNU Arch	7. Darcs	

Table of Contents

1.0 Introduction **5**

2.0 Requirement **6**

2.1 Hardware Requirements 6

2.2 Software Requirements 6

2.3 Browser Requirement 7

3.0 Comparison **8**

3.1 Advantages and Disadvantages 8

3.2 Comparison between Puppeteer and Selenium 9

4.0 Installation and Setup Procedure **10**

5.0 Puppeteer Features **14**

6.0 Screenshots Usage Example **16**

6.1 Browser Automation Example 16

6.2 Screenshot Capture Example 17

6.3 PDF Generation Example 18

6.4 Form Automation Example 19

6.5 Mouse and Keyboard Simulation Example 20

6.6 Performance Testing Example 21

7.0 Conclusion **22**

8.0 Reference **23**

1.0 Introduction

Puppeteer is an open-source automation testing tool developed by Google that is used to control and automate web browsers programmatically. Puppeteer is built for Node.js and mainly works with Chromium and Google Chrome browsers. It provides a high-level Application Programming Interface (API) that allows developers and testers to automate browser activities such as opening websites, clicking buttons, typing into forms, taking screenshots, generating PDF files, and extracting website information. Puppeteer is widely used in automation testing because it can simulate real user interactions accurately and efficiently.

Puppeteer has become one of the popular tools in modern web automation due to its simplicity and powerful features. Unlike some automation tools that require complex configuration, Puppeteer offers an easier installation and setup process through Node.js and npm. The tool supports headless browser execution, which means browser automation can run in the background without displaying the browser window. This feature helps improve testing speed and system performance. Besides automation testing, Puppeteer is also commonly used for web scraping, website monitoring, form automation, performance testing, screenshot capturing, and PDF generation.

One of the main advantages of Puppeteer is its strong integration with JavaScript and modern web technologies. Since many websites today are built using dynamic JavaScript frameworks, Puppeteer is highly suitable for testing modern web applications. The tool also provides accurate browser control, allowing testers to automate keyboard input, mouse actions, page navigation, and browser interactions effectively. Due to its flexibility, fast execution, and easy-to-understand syntax, Puppeteer is suitable for both beginners and professional developers in software testing and browser automation projects.

2.0 Requirement

Before using Puppeteer, several hardware and software requirements must be prepared to ensure the tool runs properly and efficiently. Since Puppeteer automates a real web browser, the computer system must have adequate resources and compatible software environments. Proper setup is important to avoid installation errors and improve browser automation performance.

2.1 Hardware Requirements

The recommended hardware requirements are:

- Minimum 4GB RAM
- Intel or AMD processor
- Stable internet connection
- At least 2GB available storage

A stable internet connection is important because Puppeteer downloads browser dependencies and accesses websites during automation testing.

2.2 Software Requirements

To use Puppeteer, several software components must be installed, including:

- Node.js
- npm (Node Package Manager)
- Visual Studio Code or another code editor
- Windows 10/11, Linux, or macOS operating system

Node.js is required because Puppeteer scripts are written and executed using JavaScript in the Node.js environment. npm is used to install Puppeteer and manage project dependencies. Visual Studio Code is recommended because it provides useful features such as syntax highlighting, integrated terminal, and debugging support, which help developers write and execute Puppeteer scripts more easily.

2.3 Browser Requirement

During installation, Puppeteer automatically downloads Chromium, which is the browser used for automation testing. Users may also configure Puppeteer to use Google Chrome if needed. Chromium provides stable and reliable browser automation performance for testing modern web applications.

Overall, preparing the correct requirements helps ensure that Puppeteer can run smoothly and perform browser automation tasks effectively.

3.0 Comparison

3.1 Advantages and Disadvantages

The following table shows the advantages and disadvantages of Puppeteer based on several important criteria related to automation testing and browser control.

Criteria	Advantages	Disadvantages
Installation and Setup	Easy to install using npm with simple configuration	Requires installation of Node.js and internet connection
Browser Automation	Provides accurate and fast browser automation	Mainly optimized for Chromium and Chrome browsers
Execution Speed	Faster execution compared to some automation tools	May consume high system resources during execution
Headless Testing	Supports headless mode for background execution	Debugging may be harder in headless mode
Screenshot and PDF Features	Can easily capture screenshots and generate PDF files	Advanced customization may require additional coding
Ease of Use	Simple API and beginner-friendly syntax	Beginners without coding experience may face difficulties
Performance Testing	Useful for measuring webpage loading performance	Not specialized for heavy load or stress testing
JavaScript Integration	Excellent support for modern JavaScript-based websites	Requires knowledge of JavaScript programming

3.2 Comparison between Puppeteer and Selenium

Criteria	Puppeteer	Selenium
Developer	Developed by Google	Developed by Selenium Project contributors
Programming Language	Mainly uses JavaScript with Node.js	Supports multiple languages such as Java, Python, C#, Ruby, and JavaScript
Browser support	Mainly supports Chromium and Chrome browsers	Supports multiple browsers including Chrome, Firefox, Edge, and Safari
Installation and Setup	Easier and faster setup using npm	More complex setup with WebDriver configuration
Execution Speed	Faster execution due to direct browser control	Generally slower because it communicates through WebDriver

4.0 Installation and Setup Procedure

Step 1: Install Node.js

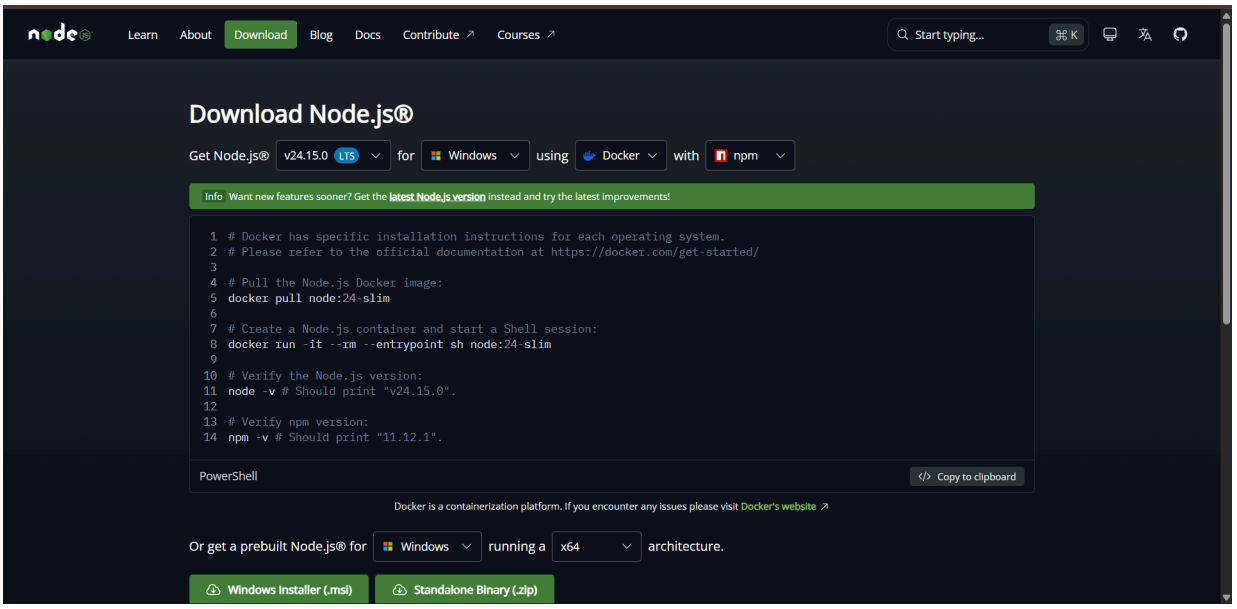


Figure 1: *Download Node.js from the official website.*

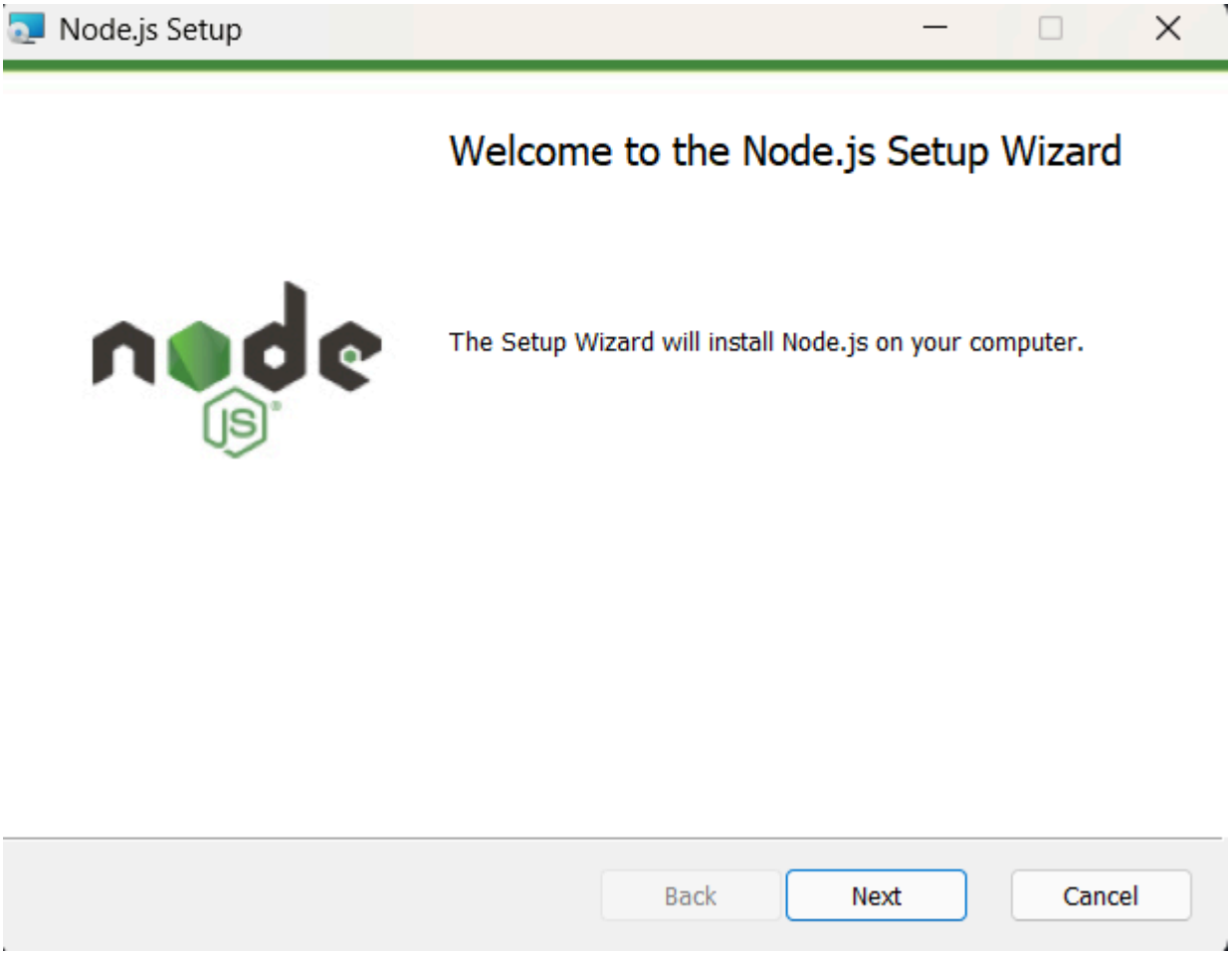


Figure 2: *Installation process of Node.js.*

Step 2: Verify Node.js and npm Installation

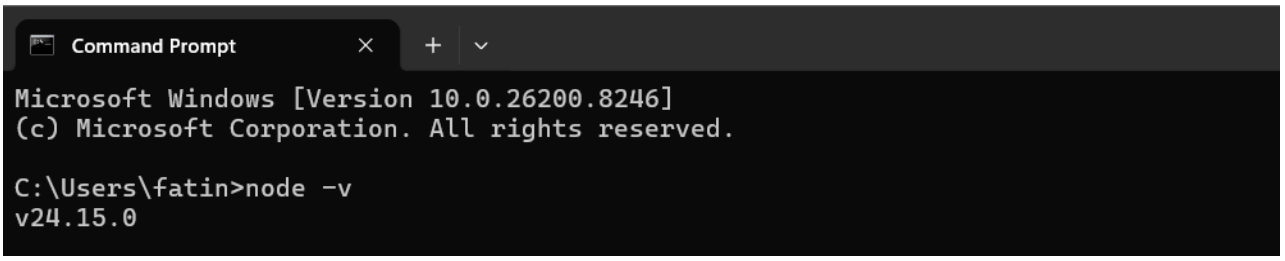


Figure 3: *Verification of Node.js using terminal commands.*



Figure 4: *Verification of npm installation using terminal commands.*

Step 3: Create Project Folder



Figure 5: *Creation of Puppeteer project folder using terminal command.*

Step 4: Initialize Node.js Project



Figure 6: *Initialization of Node.js project using the npm initialization command.*

Step 5: Install Puppeteer

```
C:\Users\fatin\puppeteer-project>npm install puppeteer

added 99 packages, and audited 100 packages in 4m

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Figure 7: *Successful installation of Puppeteer using npm package manager.*

Step 6: Create Test File

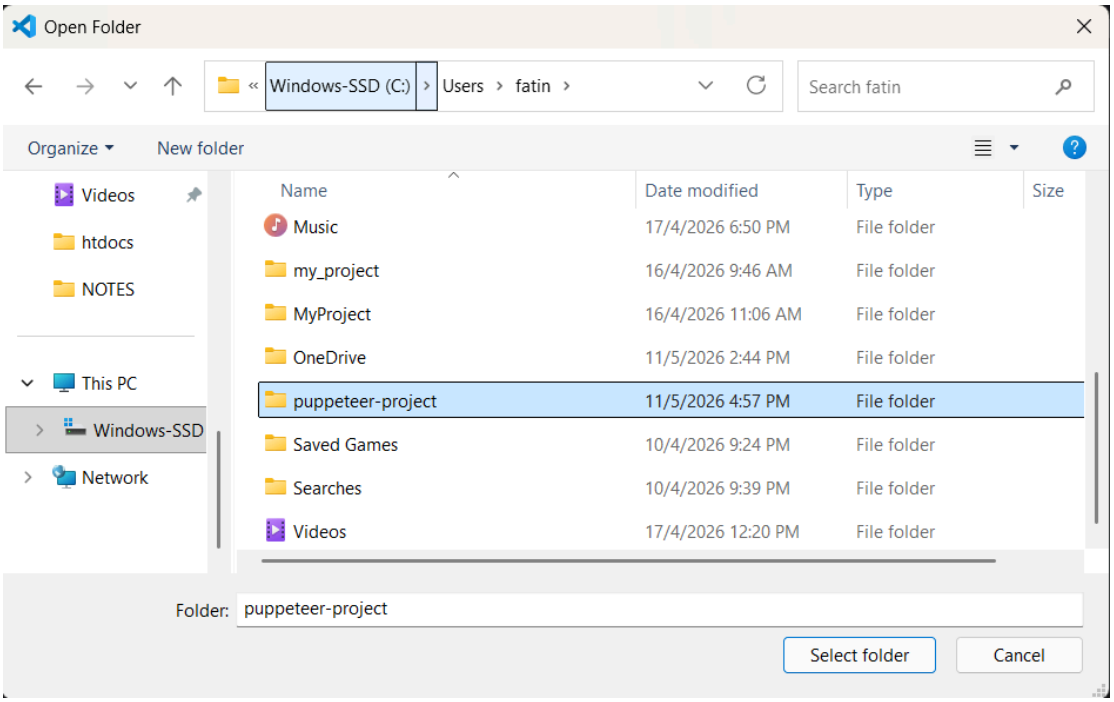


Figure 8: *Opening the Puppeteer Project folder in Visual Studio Code.*

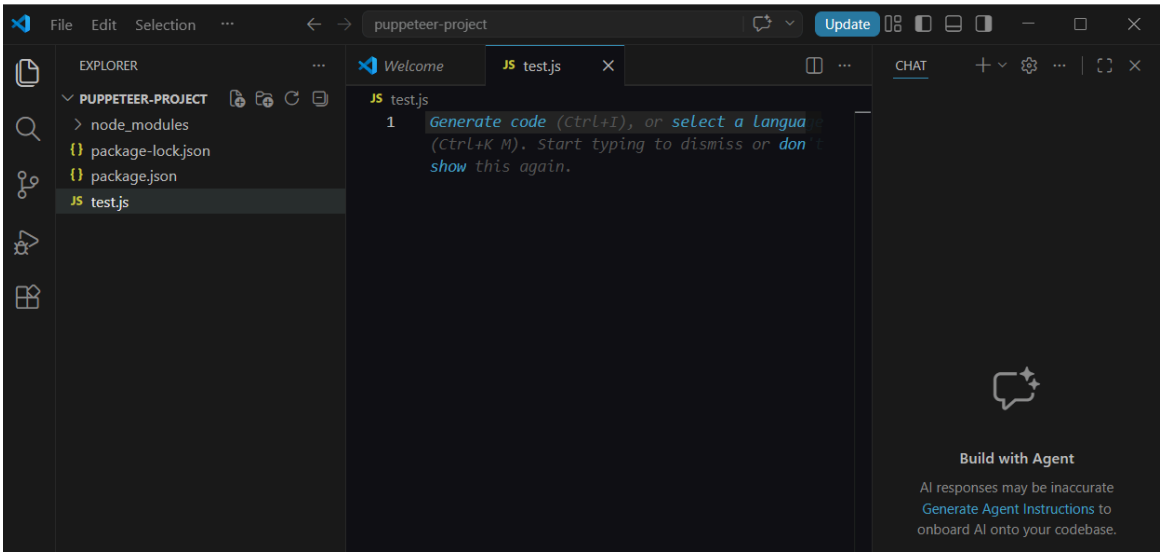


Figure 9: *Creation of the test.js file for Puppeteer automation scripting.*

Step 7: Add Sample Code

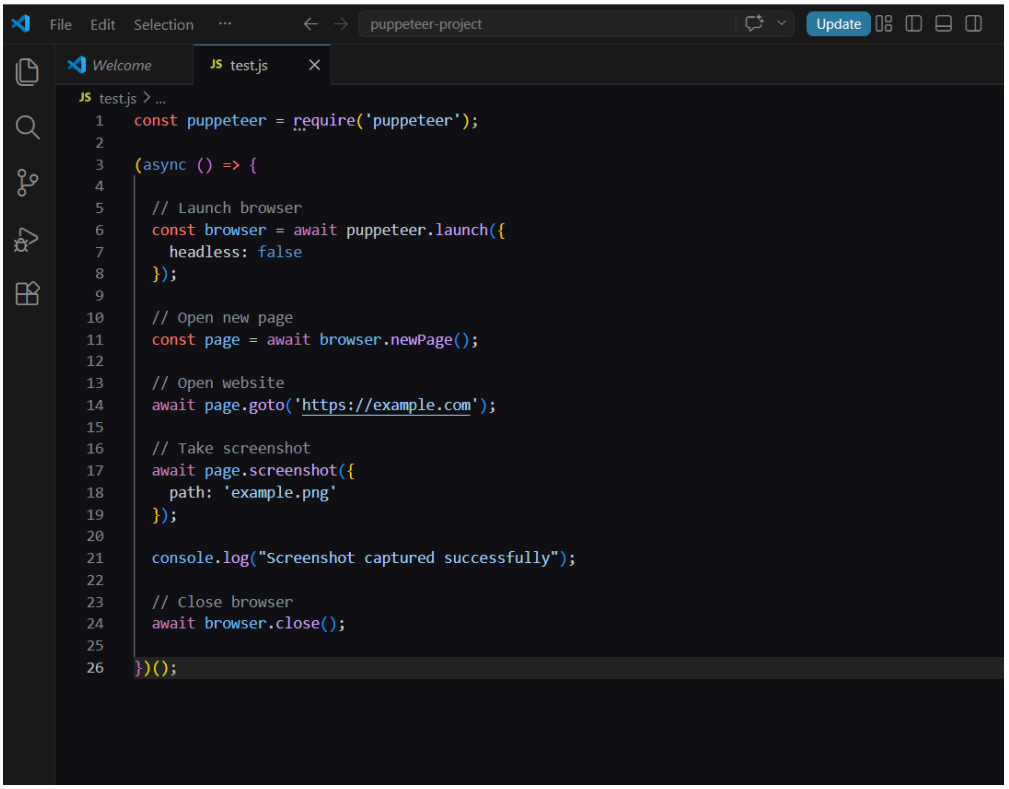


Figure 10: *Writing Puppeteer automation code inside the test.js file.*

Step 8: Run the Program

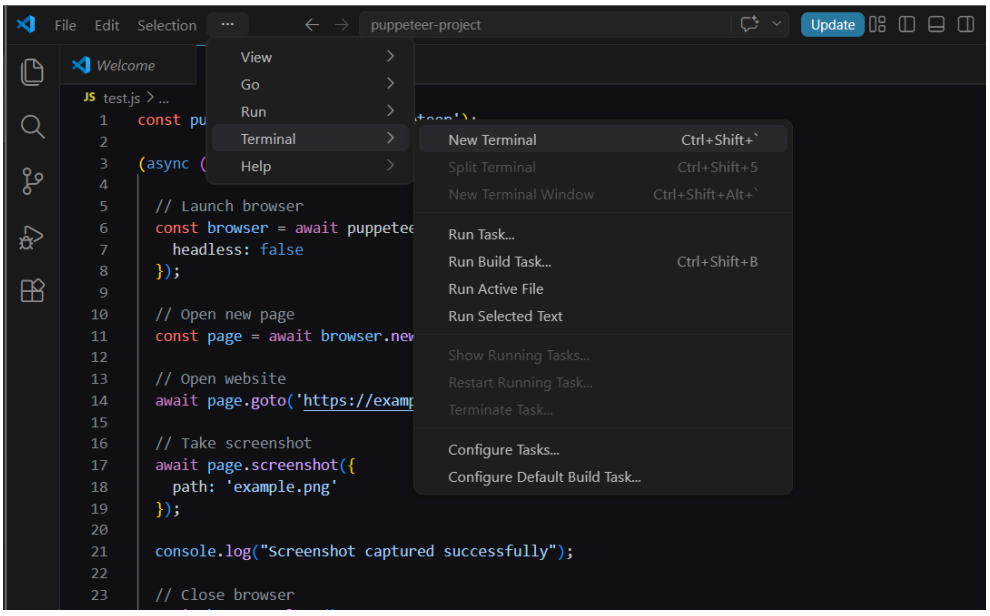


Figure 11: *Open terminal to run the code*

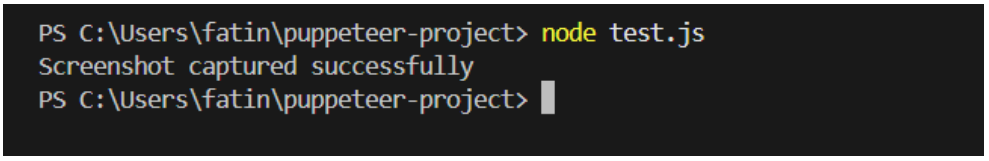


Figure 12: *Execution of Puppeteer automation script using Node.js terminal command.*

5.0 Puppeteer Features

Main Features

1. **Browser Automation**

   Puppeteer can control the browser like a real user. It can open websites, click buttons, fill forms, scroll pages, and navigate between pages. Beyond basic interactions, Puppeteer can also handle pop-ups, dropdowns, file uploads, and page redirects automatically. It uses the Chrome DevTools Protocol (CDP) to communicate directly with the browser, which gives it faster and more reliable control compared to tools that rely on WebDriver. This makes Puppeteer highly suitable for end-to-end testing of modern web applications.

2. **Screenshot Capture**

   Puppeteer can take screenshots of webpages automatically at any point during automation. It supports full-page screenshots, viewport screenshots, and screenshots of specific elements on a page. Screenshots can be saved in PNG or JPEG format with customizable quality settings. This feature is useful for UI testing to detect visual regressions, bug reporting to capture exact error states, and website monitoring to track page appearance over time.

3. **PDF Generation**

   Puppeteer can convert any webpage into a PDF file programmatically. It supports customization of paper size, page orientation, margins, header, footer, and print media settings. This feature is commonly used for generating invoices, reports, and printable documents directly from web content. Since Puppeteer renders the page using a real Chromium engine, the generated PDF accurately reflects the actual appearance of the webpage including styles, images, and dynamic content.

4. **Form Automation**

Puppeteer can automatically fill in form fields and submit them without any manual input. It can type text into input fields, select dropdown options, check checkboxes, and click submit buttons. This feature is useful for testing login forms, registration pages, search functions, and any repetitive data entry tasks. Puppeteer can also handle form validations and detect error messages after submission, making it ideal for thorough form testing workflows.

5. **Keyboard and Mouse Simulation**

Puppeteer can simulate real keyboard typing and mouse movements, including hover effects, drag-and-drop actions, double clicks, right clicks, and keyboard shortcuts. This level of input simulation allows testers to replicate complex user interactions that would otherwise be difficult to automate. Testing interactive UI components such as sliders, drag-and-drop widgets, and context menus becomes straightforward with this feature.

6. **Performance Testing**

Puppeteer can measure webpage loading performance by collecting metrics such as page load time, DOM content load time, first paint time, and resource loading durations. It integrates with Chrome DevTools Protocol to capture detailed performance traces and network activity logs. This data helps developers identify slow-loading resources, render-blocking scripts, and performance bottlenecks in their web applications. While Puppeteer is not a dedicated load testing tool, it is effective for measuring frontend performance of individual pages.

6.0 Screenshots Usage Example

6.1 Browser Automation Example

1. Code

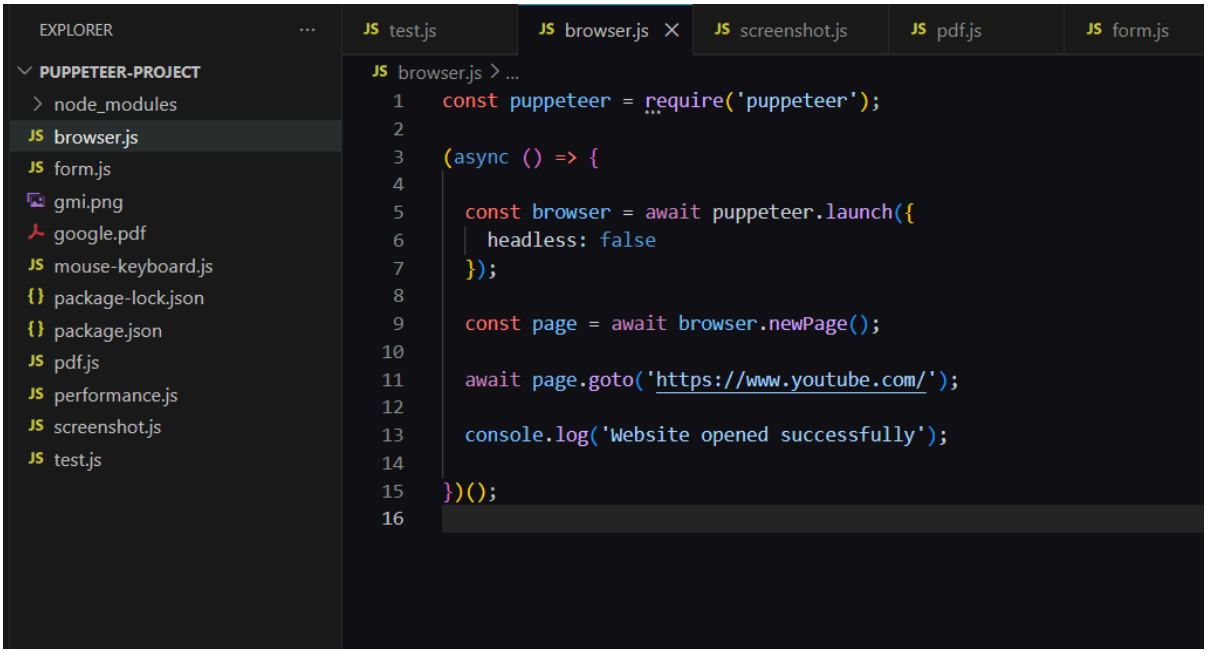


Figure 13: *Writing browser automation code inside browser.js file.*

2. Command

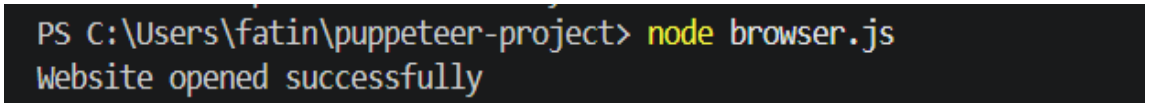


Figure 14: *Execution of browser automation code using terminal command.*

3. Output

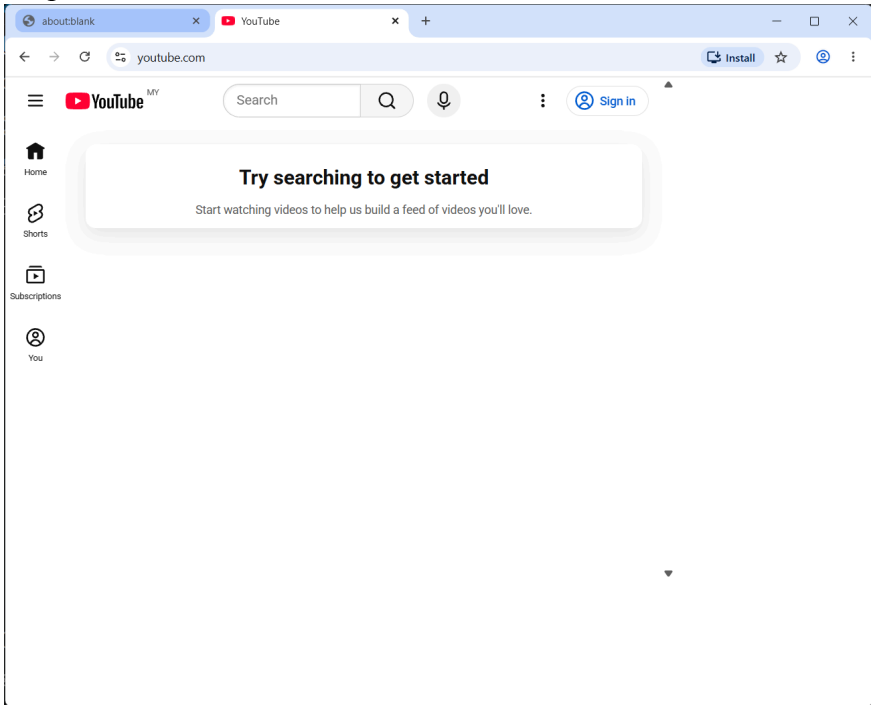


Figure 15: *Automated Chromium browser launched successfully by Puppeteer.*

6.2 Screenshot Capture Example

1. Code

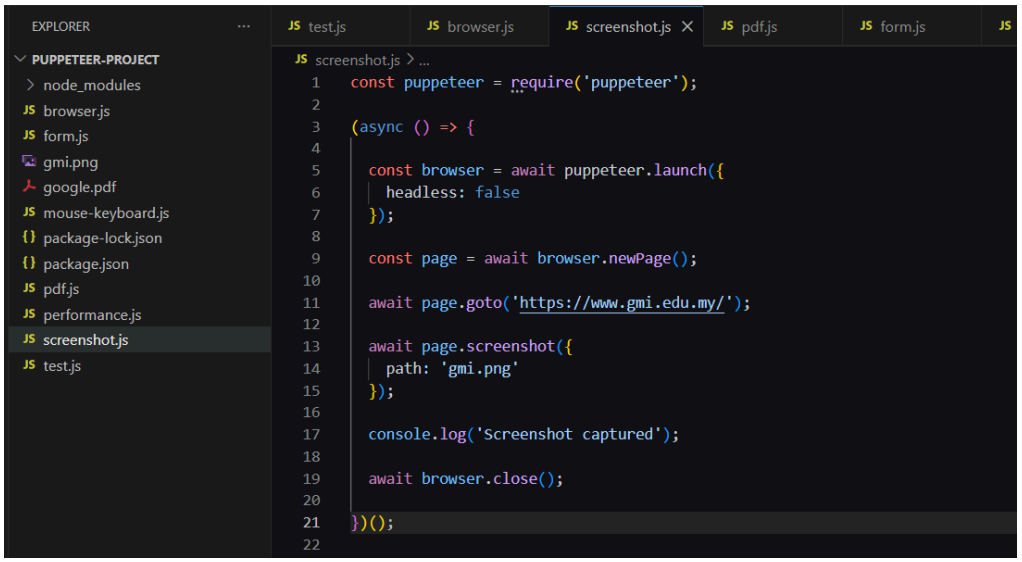


Figure 16: *Writing screenshot capture code inside screenshot.js file.*

2. Command

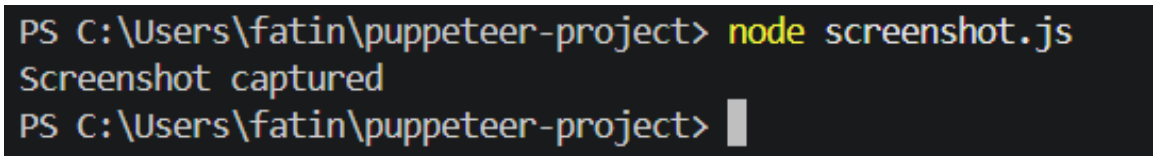


Figure 17: *Execution of screenshot capture code using terminal command.*

3. Output

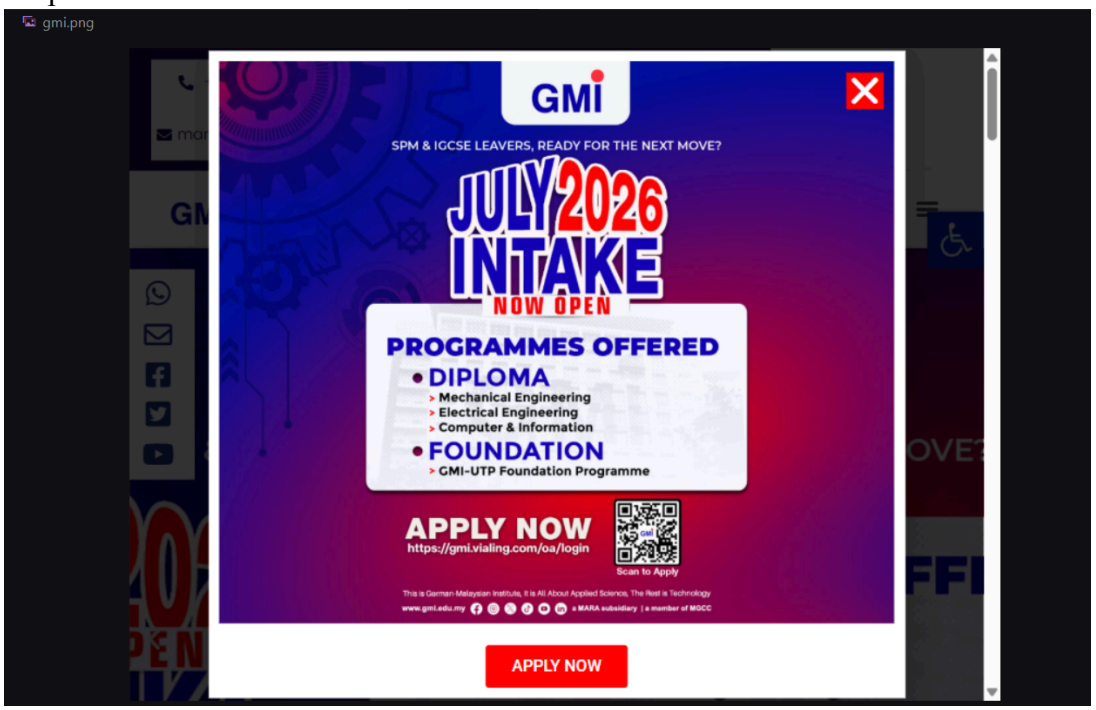


Figure 18: *Screenshot image generated automatically after successful Puppeteer execution.*

6.3 PDF Generation Example

1. Code

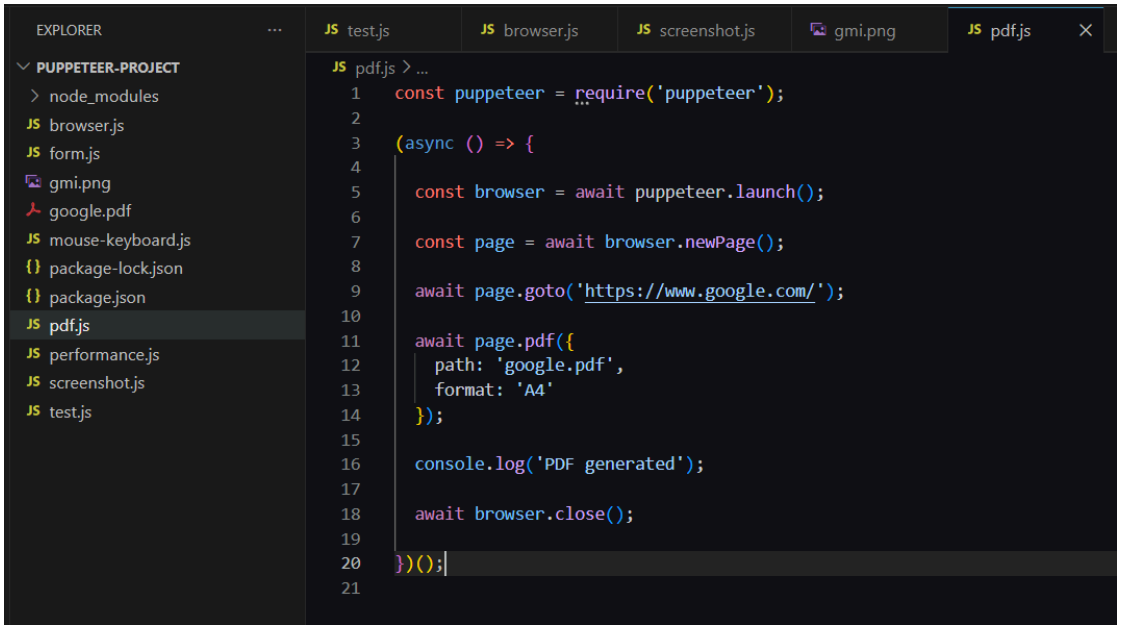


Figure 19: *Writing PDF generation code inside pdf.js file.*

2. Command

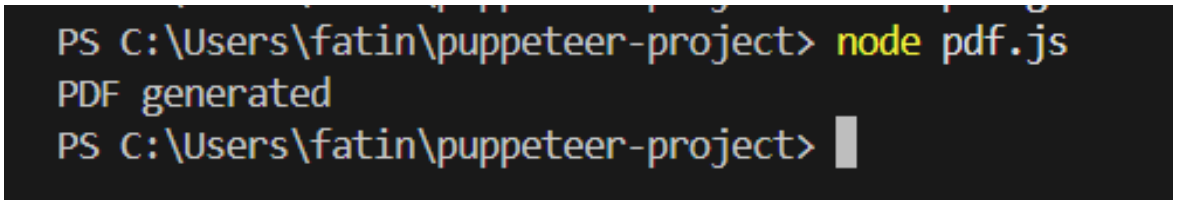


Figure 20: *Execution of PDF generation code using terminal command.*

3. Output

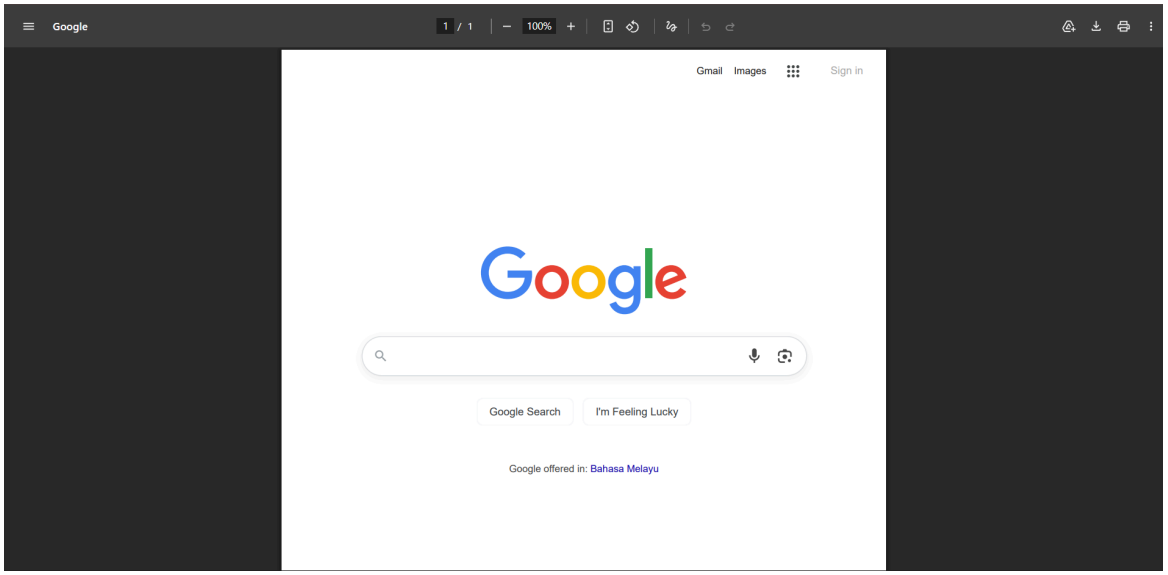


Figure 21: *PDF file generated automatically using Puppeteer PDF generation feature.*

6.4 Form Automation Example

1. Code

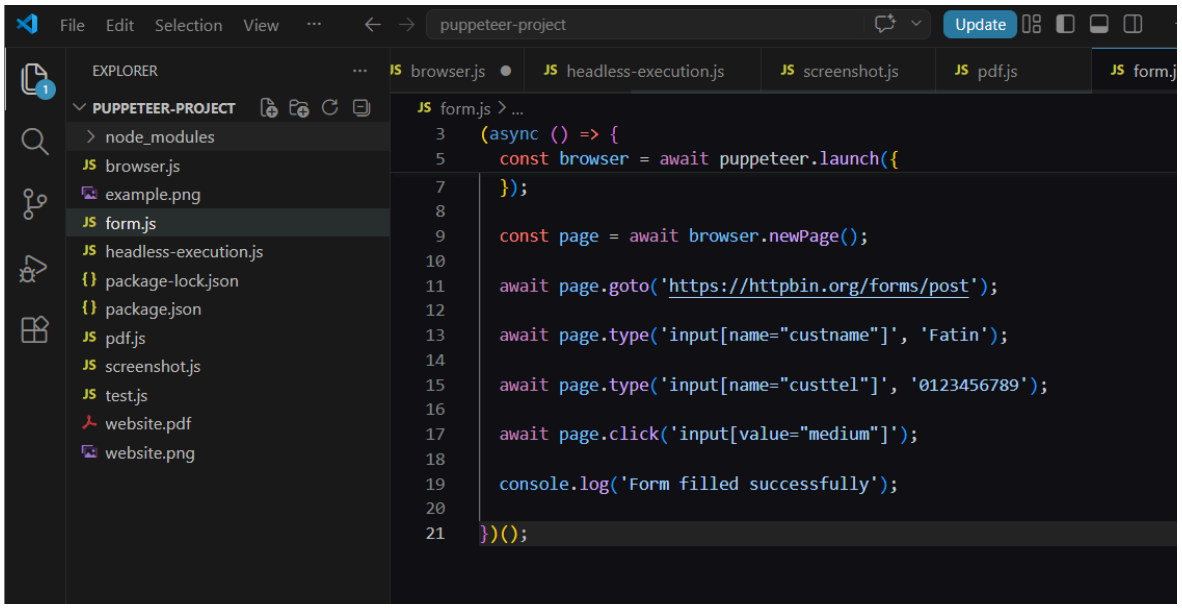


Figure 22: *Writing form automation code inside form.js file.*

2. Command

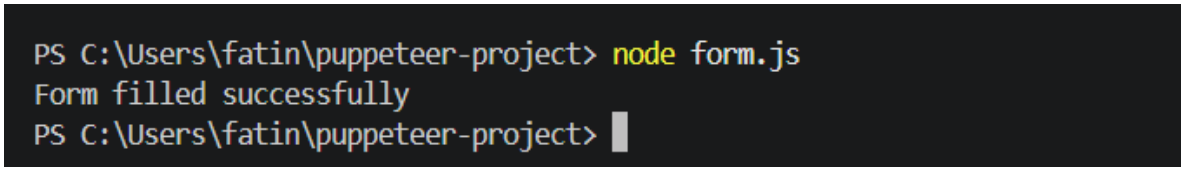


Figure 23: *Execution of form automation code using terminal command.*

3. Output

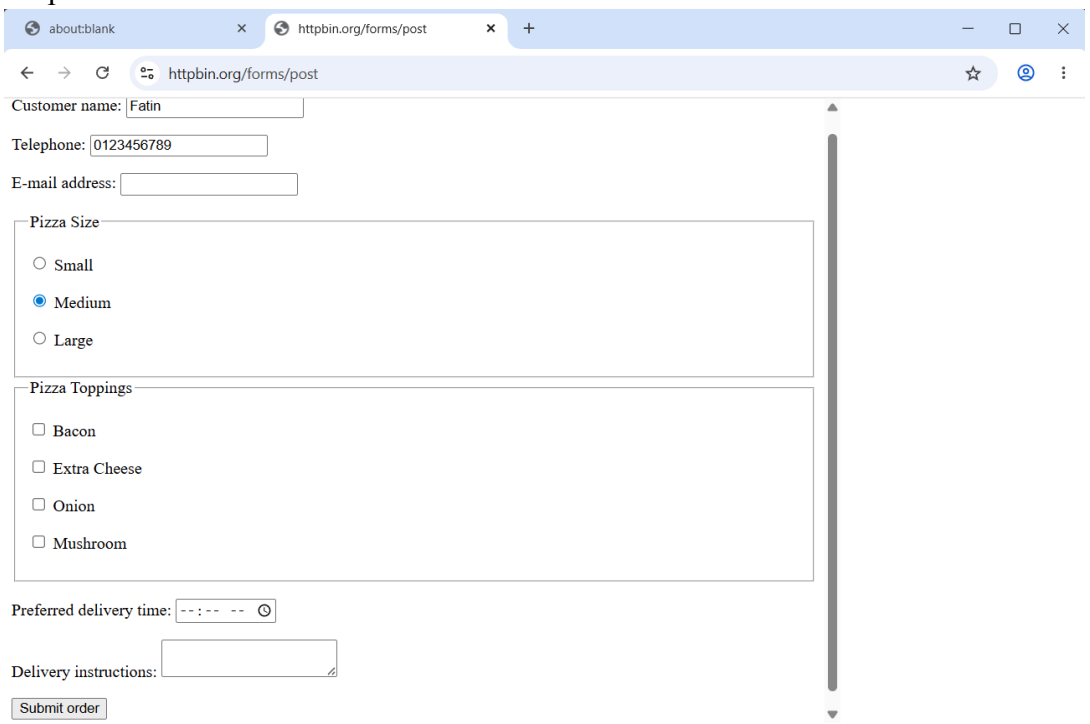


Figure 24: *Automated form filling process performed using Puppeteer form automation feature.*

6.5 Mouse and Keyboard Simulation Example

1. Code

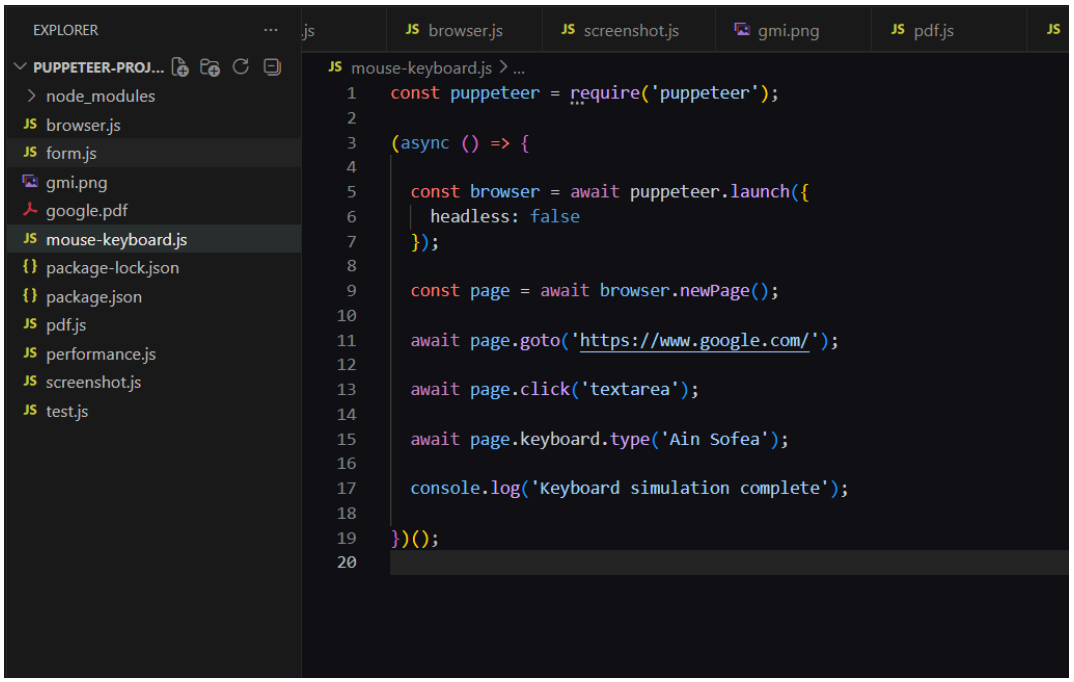


Figure 25: *Writing mouse and keyboard simulation code inside mouse-keyborard.js file.*

2. Command

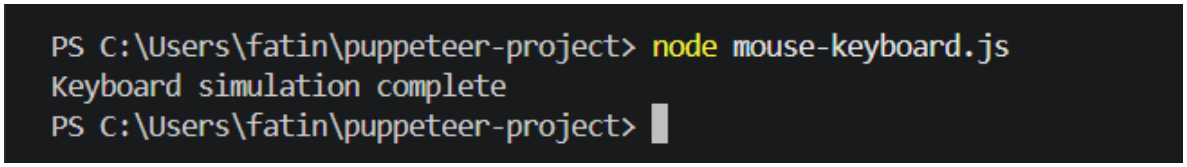


Figure 26: *Execution of mouse and keyboard simulation code using terminal command.*

3. Output

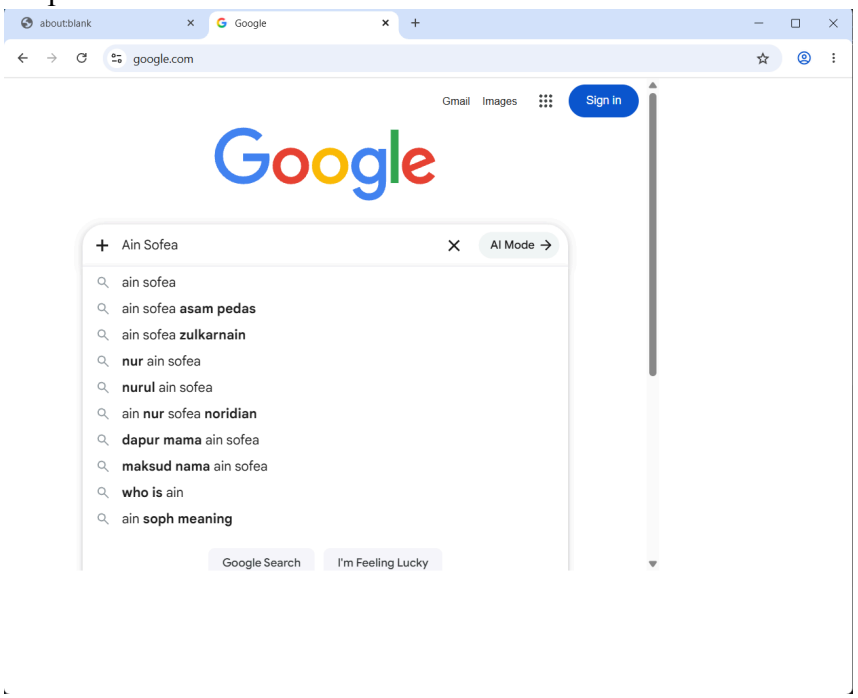


Figure 27: *Puppeteer script performing automated mouse click and keyboard typing simulation on a web browser.*

6.6 Performance Testing Example

1. Code

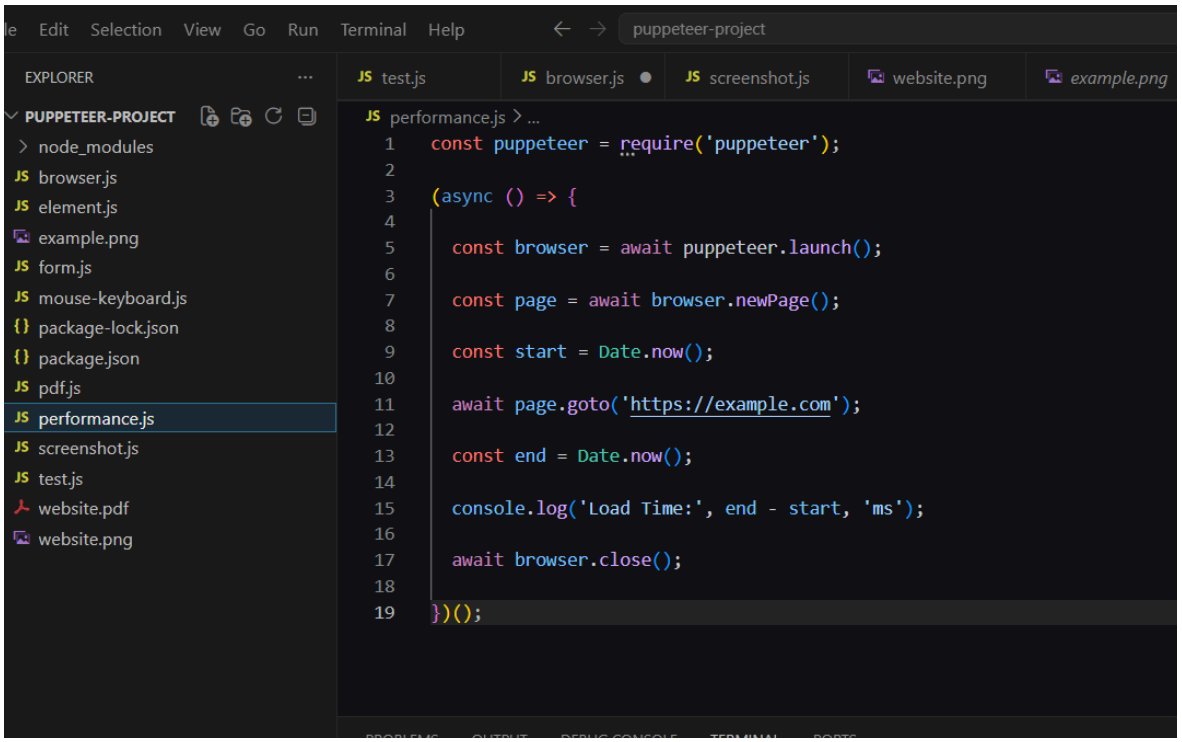


Figure 28: *Writing performance testing code inside performance.js file.*

2. Command and Output

```
PS C:\Users\fatin\puppeteer-project> node performance.js
Load Time: 289 ms
PS C:\Users\fatin\puppeteer-project>
```

Figure 29: *Terminal output displaying webpage loading time measurement generated by Puppeteer.*

7.0 Conclusion

Puppeteer is a powerful and modern open-source automation testing tool developed by Google that has significantly simplified the way developers and testers approach browser automation. Built on Node.js and powered by the Chrome DevTools Protocol, Puppeteer provides direct and reliable control over Chromium and Google Chrome browsers, making it one of the most efficient tools available for web automation tasks today.

Throughout this report, Puppeteer has been examined in terms of its requirements, advantages, disadvantages, installation procedure, features, and practical usage examples. From the discussion, it is clear that Puppeteer offers a wide range of capabilities that go beyond simple browser automation. Its ability to perform headless testing, capture screenshots, generate PDF files, simulate keyboard and mouse interactions, scrape dynamic web content, and measure page performance makes it a versatile tool suitable for many different use cases in software testing and web development.

One of the key strengths of Puppeteer is its ease of setup and use. With just a few commands using npm, developers can install Puppeteer and begin writing automation scripts immediately without complex configurations. Its straightforward JavaScript-based API is beginner-friendly while still being powerful enough for advanced professional use cases. This makes Puppeteer accessible to a wide range of users, from students learning automation testing to experienced software engineers working on large-scale web projects.

Despite its limitations, Puppeteer remains an excellent choice for projects that primarily target Chrome or Chromium environments, which represent the majority of web users today. Its active development and strong community support from Google ensure that Puppeteer continues to receive updates, improvements, and new features over time.

In conclusion, Puppeteer is a highly recommended automation testing tool for web developers and software testers looking for a fast, reliable, and easy-to-use solution for browser automation. It is suitable for a variety of testing scenarios including functional testing, UI regression testing, performance testing, and web scraping. With its modern JavaScript integration, powerful feature set, and growing adoption in the industry, Puppeteer is well-positioned as a valuable tool in any software testing toolkit.

8.0 Reference

Puppeteer Guides | Puppeteer. (n.d.). Pptr.dev. https://pptr.dev/category/guides

Puppeteer | Puppeteer. (n.d.). Pptr.dev. https://pptr.dev

puppeteer. (n.d.). Npm. https://www.npmjs.com/package/puppeteer

Documentation. (n.d.). Node.js. https://nodejs.org/en/docs

SeleniumHQ Browser Automation. (n.d.). Www.selenium.dev. https://www.selenium.dev

Puppeteer. (2022, February 4). GitHub. https://github.com/puppeteer/puppeteer